

```
In [ ]: # 2.2 Линейная регрессия.
```

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
```

```
In [ ]: ## 1. Загрузка и обработка датасета.
Датасет, его загрузка и обработка как в задании 1.2.
```

```
In [33]: df_air = pd.read_csv(
    'C:/Users/Elizaveta/Downloads/air_quality/AirQuality.csv',
    delimiter=';',
    decimal=',',
    na_values=[' ', 'NA', 'nan', -200]
)
print(f"размер после загрузки: {df_air.shape}")
print("столбцы после загрузки:", list(df_air.columns))
df_air = df_air.loc[:, ~df_air.columns.str.contains('Unnamed')]
print(f"\nразмер таблицы после удаления пустых столбцов: {df_air.shape}")
print("столбцы:", list(df_air.columns))
print(df_air.head())
print(df_air.info())
print(df_air.isnull().sum())
```

размер после загрузки: (9471, 17)

столбцы после загрузки: ['Date', 'Time', 'CO(GT)', 'PT08.S1(CO)', 'NMHC(GT)', 'C6H6(GT)', 'PT08.S2(NMHC)', 'NOx(GT)', 'PT08.S3(NOx)', 'NO2(GT)', 'PT08.S4(NO2)', 'PT08.S5(O3)', 'T', 'RH', 'AH', 'Unnamed: 15', 'Unnamed: 16']

размер таблицы после удаления пустых столбцов: (9471, 15)

столбцы: ['Date', 'Time', 'CO(GT)', 'PT08.S1(CO)', 'NMHC(GT)', 'C6H6(GT)', 'PT08.S2(NMHC)', 'NOx(GT)', 'PT08.S3(NOx)', 'NO2(GT)', 'PT08.S4(NO2)', 'PT08.S5(O3)', 'T', 'RH', 'AH']

	Date	Time	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	\
0	10/03/2004	18.00.00	2.6	1360.0	150.0	11.9	
1	10/03/2004	19.00.00	2.0	1292.0	112.0	9.4	
2	10/03/2004	20.00.00	2.2	1402.0	88.0	9.0	
3	10/03/2004	21.00.00	2.2	1376.0	80.0	9.2	
4	10/03/2004	22.00.00	1.6	1272.0	51.0	6.5	

	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08.S5(O3)	\
0	1046.0	166.0	1056.0	113.0	1692.0	1268.0	
1	955.0	103.0	1174.0	92.0	1559.0	972.0	
2	939.0	131.0	1140.0	114.0	1555.0	1074.0	
3	948.0	172.0	1092.0	122.0	1584.0	1203.0	
4	836.0	131.0	1205.0	116.0	1490.0	1110.0	

	T	RH	AH
0	13.6	48.9	0.7578
1	13.3	47.7	0.7255
2	11.9	54.0	0.7502
3	11.0	60.0	0.7867
4	11.2	59.6	0.7888

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 9471 entries, 0 to 9470

Data columns (total 15 columns):

#	Column	Non-Null Count	Dtype
0	Date	9357 non-null	object
1	Time	9357 non-null	object
2	CO(GT)	7674 non-null	float64
3	PT08.S1(CO)	8991 non-null	float64
4	NMHC(GT)	914 non-null	float64
5	C6H6(GT)	8991 non-null	float64
6	PT08.S2(NMHC)	8991 non-null	float64
7	NOx(GT)	7718 non-null	float64
8	PT08.S3(NOx)	8991 non-null	float64
9	NO2(GT)	7715 non-null	float64
10	PT08.S4(NO2)	8991 non-null	float64
11	PT08.S5(O3)	8991 non-null	float64
12	T	8991 non-null	float64
13	RH	8991 non-null	float64
14	AH	8991 non-null	float64

dtypes: float64(13), object(2)

memory usage: 1.1+ MB

None

Date 114

Time 114

CO(GT) 1797

PT08.S1(CO) 480

NMHC(GT) 8557

C6H6(GT) 480

PT08.S2(NMHC) 480

NOx(GT) 1753

```
PT08.S3(NOx)      480
NO2(GT)           1756
PT08.S4(NO2)      480
PT08.S5(O3)       480
T                 480
RH                480
AH                480
dtype: int64
```

```
In [35]: X = df_air[feature_columns]
Y = df_air[target_column]
mask = X.notnull().all(axis=1)&Y.notnull()
X_clean = X[mask]
Y_clean = Y[mask]
print(f"Размер X: {X_clean.shape}")
print(f"Размер Y: {Y_clean.shape}")
print(f"\nЗначения целевой переменной {target_column}:")
print(f"Min: {Y_clean.min():.3f}")
print(f"Max: {Y_clean.max():.3f}")
print(f"Среднее: {Y_clean.mean():.3f}")
print(f"Медиана: {Y_clean.median():.3f}")
print(f"Стандартное отклонение: {Y_clean.std():.3f}")
```

```
Размер X: (6941, 11)
Размер Y: (6941,)
```

```
Значения целевой переменной C6H6(GT):
Min: 0.200
Max: 63.700
Среднее: 10.554
Медиана: 8.800
Стандартное отклонение: 7.465
```

```
In [36]: from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X_clean, Y_clean,
                                                    test_size=0.3,
                                                    random_state=42)

print(f"Обучающая выборка: {X_train.shape}")
print(f"Тестовая выборка: {X_test.shape}")
```

```
Обучающая выборка: (4858, 11)
Тестовая выборка: (2083, 11)
```

```
In [ ]: ## 2. Создание бейзлайна.
```

Ниже представлено построение базовой модели линейной регрессии LinearRegression. Обучающая выборка (train) используется для обучения модели. Тестовая выборка (test) используется для оценки качества обученной модели. Используются метрики MAE, RMSE, R2. MAE=0.7759 показывает среднюю величину ошибки в тех же единицах, что и целевая переменная. RMSE=1.1479 также показывает среднюю ошибку переменной, но за счёт возведения в квадрат демонстрирует наличие выбросов. R2=0.0.9769 показывает долю дисперсии целевой переменной, полученное значение по базовой модели демонстрирует довольно высокие показатели. Она служит отправной точкой для построения более сложных моделей.

```
In [10]: from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
lin_reg_baseline = LinearRegression()
lin_reg_baseline.fit(X_train, Y_train)
Y_baseline_prediction = lin_reg_baseline.predict(X_test)
MAE_baseline = mean_absolute_error(Y_test, Y_baseline_prediction)
RMSE_baseline = np.sqrt(mean_squared_error(Y_test, Y_baseline_prediction))
```

```
R2_baseline = r2_score(Y_test, Y_baseline_prediction)
print(f"\n baseline LinearRegression:")
print(f"MAE: {MAE_baseline:.4f}")
print(f"RMSE: {RMSE_baseline:.4f}")
print(f"R2: {R2_baseline:.4f}")
```

```
baseline LinearRegression:
MAE: 0.7759
RMSE: 1.1479
R2: 0.9769
```

In []: *## 3. Улучшение бейзлайна.*

Гипотеза 1: Регуляризация и масштабирование признаков.
 Проведена регуляризация линейной регрессии с автоматическим подбором коэффициента. Перед этим признаки приводятся к единому масштабу с помощью StandardScaler, т.к. некорректно. Создаём сетку из 20 значений коэффициента регуляризации alpha от 0. очень сильную регуляризацию. RidgeCV автоматически выполняет 5-кратную кросс-вал наименьшую среднеквадратичную ошибку. Полученный best alpha: 0.3360. Регуляриза

```
In [37]: from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
alphas = np.logspace(-3, 3, 20)
ridge_cv = RidgeCV(alphas=alphas)
ridge_cv.fit(X_train_scaled, Y_train)
best_alpha = ridge_cv.alpha_
print(f"best alpha: {best_alpha:.4f}")
Y_prediction_ridge_cv = ridge_cv.predict(X_test_scaled)
MAE_ridge = mean_absolute_error(Y_test, Y_prediction_ridge_cv)
RMSE_ridge = np.sqrt(mean_squared_error(Y_test, Y_prediction_ridge_cv))
R2_ridge = r2_score(Y_test, Y_prediction_ridge_cv)
print(f" \nRidge scaled lin reg:")
print(f" MAE: {MAE_ridge:.4f}")
print(f" RMSE: {RMSE_ridge:.4f}")
print(f" R2: {R2_ridge:.4f}")
```

```
best alpha: 0.3360
```

```
Ridge scaled lin reg:
MAE: 0.7758
RMSE: 1.1483
R2: 0.9769
```

In []: Гипотеза 2: RobustScaler и HuberRegressor.

Масштабирование RobustScaler устойчиво к выбросам, в отличие от обычной линейной HuberRegressor обладает модифицированной функцией потерь, которая ведёт себя как (параметр epsilon=1.35 определяет порог, после которого потери становятся линейными). Произошло уменьшение размера ошибки MAE=0.7461 (было MAE=0.7758), улучшение того (было RMSE=1.1483), т.е. выбросы оказывают меньшее влияние. R2 практически не изменился. Таким образом, качество модели улучшилось за счёт адаптации к аномалиям.

```
In [39]: from sklearn.preprocessing import RobustScaler
from sklearn.linear_model import HuberRegressor
robust_scaler = RobustScaler()
X_train_robust = robust_scaler.fit_transform(X_train)
X_test_robust = robust_scaler.transform(X_test)

huber = HuberRegressor(max_iter=1000)
```

```

huber.fit(X_train_robust, Y_train)
Y_prediction_huber = huber.predict(X_test_robust)
MAE_huber = mean_absolute_error(Y_test, Y_prediction_huber)
RMSE_huber = np.sqrt(mean_squared_error(Y_test, Y_prediction_huber))
R2_huber = r2_score(Y_test, Y_prediction_huber)
print(f" \nRobust huber lin reg:")
print(f" MAE: {MAE_huber:.4f}")
print(f" RMSE: {RMSE_huber:.4f}")
print(f" R2: {R2_huber:.4f}")

```

Robust huber lin reg:

MAE: 0.7461
RMSE: 0.8638
R2: 0.9732

In []: Гипотеза 3: Полиномиальные признаки.
Расширение исходного пространства признаков за счёт их степеней и взаимодействий продемонстрировало аномально высокие метрики, что может говорить о переобучении.
best alpha for poly: 0.0785
Ridge poly lin reg:
MAE: 0.0370
RMSE: 0.0531
R2: 1.0000

```

In [43]: from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)
scaler_poly = StandardScaler()
X_train_poly_scaled = scaler_poly.fit_transform(X_train_poly)
X_test_poly_scaled = scaler_poly.transform(X_test_poly)
alphas = np.logspace(-3, 3, 20)
ridge_poly = RidgeCV(alphas=alphas)
ridge_poly.fit(X_train_poly_scaled, Y_train)
best_alpha_poly = ridge_poly.alpha_
print(f"best alpha for poly: {best_alpha_poly:.4f}")
Y_prediction_poly = ridge_poly.predict(X_test_poly_scaled)
MAE_poly = mean_absolute_error(Y_test, Y_prediction_poly)
RMSE_poly = np.sqrt(mean_squared_error(Y_test, Y_prediction_poly))
R2_poly = r2_score(Y_test, Y_prediction_poly)
print(f" \nRidge poly lin reg:")
print(f" MAE: {MAE_poly:.4f}")
print(f" RMSE: {RMSE_poly:.4f}")
print(f" R2: {R2_poly:.4f}")

```

best alpha for poly: 0.0785

Ridge poly lin reg:

MAE: 0.0370
RMSE: 0.0531
R2: 1.0000

In []: Ручной перебор значений параметра регуляризации для модели, обученной на полином и переобучением.

```

In [25]: alphas_to_try = [0.1, 0.5, 1.0, 5.0, 10.0, 50.0, 100.0]
results = []
for alpha in alphas_to_try:
    ridge = Ridge(alpha=alpha)

```

```

ridge.fit(X_train_poly_scaled, Y_train)
y_train_pred = ridge.predict(X_train_poly_scaled)
y_test_pred = ridge.predict(X_test_poly_scaled)
train_r2 = r2_score(Y_train, y_train_pred)
test_r2 = r2_score(Y_test, y_test_pred)
train_mae = mean_absolute_error(Y_train, y_train_pred)
test_mae = mean_absolute_error(Y_test, y_test_pred)
results.append({
    'alpha': alpha,
    'train_r2': train_r2,
    'test_r2': test_r2,
    'train_mae': train_mae,
    'test_mae': test_mae,
    'diff_r2': train_r2 - test_r2
})
print(f"alpha={alpha:.2f}: train R2={train_r2:.6f}, test R2={test_r2:.6f}, d
best = min(results, key=lambda x: x['diff_r2'] if x['test_r2'] >= 0.99 else 1e6)
print(f"\nЛучший компромисс: alpha={best['alpha']:.2f}, test R2={best['test_r2']}

```

```

alpha=0.10: train R2=0.979478, test R2=0.976585, diff=0.002893
alpha=0.50: train R2=0.979478, test R2=0.976567, diff=0.002910
alpha=1.00: train R2=0.979477, test R2=0.976545, diff=0.002932
alpha=5.00: train R2=0.979460, test R2=0.976359, diff=0.003101
alpha=10.00: train R2=0.979411, test R2=0.976110, diff=0.003301
alpha=50.00: train R2=0.978520, test R2=0.973943, diff=0.004578
alpha=100.00: train R2=0.977133, test R2=0.971470, diff=0.005663

```

Лучший компромисс: alpha=0.10, test R2=0.976585, diff=0.002893

```

In [ ]: Гипотеза 4: Lasso-регрессия.
Используем Lasso-регрессию (L1 регуляризация) с автоматическим подбором коэффици
коэффициенты неинформативных признаков.
Best alpha: 0.000100
Train: MAE=0.7718, RMSE=1.0640, R2=0.9795
Test: MAE=0.7671, RMSE=1.1554, R2=0.9766
Разница R2: 0.002890
Число ненулевых коэффициентов: 11 из 11
Выяснилось, что полиномиальные признаки не приводят к улучшению качества модели,

```

```

In [26]: from sklearn.linear_model import LassoCV
alphas = np.logspace(-4, 2, 50)
lasso_cv = LassoCV(alphas=alphas, cv=5, max_iter=10000, random_state=42)
lasso_cv.fit(X_train_poly_scaled, Y_train)
best_alpha = lasso_cv.alpha_
print(f"Best alpha: {best_alpha:.6f}")
y_train_pred = lasso_cv.predict(X_train_poly_scaled)
y_test_pred = lasso_cv.predict(X_test_poly_scaled)
mae_train = mean_absolute_error(Y_train, y_train_pred)
rmse_train = np.sqrt(mean_squared_error(Y_train, y_train_pred))
r2_train = r2_score(Y_train, y_train_pred)
mae_test = mean_absolute_error(Y_test, y_test_pred)
rmse_test = np.sqrt(mean_squared_error(Y_test, y_test_pred))
r2_test = r2_score(Y_test, y_test_pred)
print("Train: MAE={:.4f}, RMSE={:.4f}, R2={:.4f}".format(mae_train, rmse_train,
print("Test: MAE={:.4f}, RMSE={:.4f}, R2={:.4f}".format(mae_test, rmse_test, r2
print(f"Разница R2: {r2_train - r2_test:.6f}")
n_nonzero = np.sum(lasso_cv.coef_ != 0)
print(f"Число ненулевых коэффициентов: {n_nonzero} из {len(lasso_cv.coef_)}")

```

Best alpha: 0.000100
 Train: MAE=0.7718, RMSE=1.0640, R2=0.9795
 Test: MAE=0.7671, RMSE=1.1554, R2=0.9766
 Разница R2: 0.002890
 Число ненулевых коэффициентов: 11 из 11

In []: *## 4. Имплементация алгоритма машинного обучения.*
 Был создан собственный алгоритм линейной регрессии MyLinearRegression с поддержкой
 __init__ - задание гиперпараметров.
 fit - сохранение обучающих данных.
 predict - формирование предсказания.
 Результаты:
 MyLinearRegression (raw):
 R2 = 0.9769
 MAE = 0.7759
 RMSE= 1.1479

 sklearn LinearRegression:
 R2 = 0.9769
 MAE = 0.7758
 RMSE= 1.1483
 Идентично базовой модели, что говорит о корректности алгоритма.

```
In [45]: class MyLinearRegression:
    def __init__(self, alpha=0.0, fit_intercept=True):
        self.alpha = alpha
        self.fit_intercept = fit_intercept
        self.coef_ = None
        self.intercept_ = None
    def fit(self, X, y):
        if self.fit_intercept:
            X_aug = np.c_[np.ones(X.shape[0]), X]
        else:
            X_aug = X
        n_features = X_aug.shape[1]
        reg_matrix = self.alpha * np.eye(n_features)
        if self.fit_intercept:
            reg_matrix[0, 0] = 0
        XtX = X_aug.T @ X_aug
        XtX_reg = XtX + reg_matrix
        XtX_reg_inv = np.linalg.inv(XtX_reg)
        w = XtX_reg_inv @ X_aug.T @ y
        if self.fit_intercept:
            self.intercept_ = w[0]
            self.coef_ = w[1:]
        else:
            self.coef_ = w
            self.intercept_ = 0.0
        return self
    def predict(self, X):
        if self.fit_intercept:
            return X @ self.coef_ + self.intercept_
        else:
            return X @ self.coef_

my_lr = MyLinearRegression(alpha=0.0)
my_lr.fit(X_train, Y_train)
y_pred = my_lr.predict(X_test)
print("MyLinearRegression (raw):")
print(f" R2 = {r2_score(Y_test, y_pred):.4f}")
print(f" MAE = {mean_absolute_error(Y_test, y_pred):.4f}")
```

```
print(f" RMSE= {np.sqrt(mean_squared_error(Y_test, y_pred)):.4f}")
print("\n sklearn LinearRegression:")
print(f" R2 = {r2_score(Y_test, y_pred_sk):.4f}")
print(f" MAE = {mean_absolute_error(Y_test, y_pred_sk):.4f}")
print(f" RMSE= {np.sqrt(mean_squared_error(Y_test, y_pred_sk)):.4f}")
```

MyLinearRegression (raw):

```
R2 = 0.9769
MAE = 0.7759
RMSE= 1.1479
```

sklearn LinearRegression:

```
R2 = 0.9769
MAE = 0.7758
RMSE= 1.1483
```

In []: Сравнивается работа собственной модели и базовой на масштабированных признаках и Полное совпадение метрик говорит о том, что собственный алгоритм реализован корректно.
MyLinearRegression (scaled, alpha=0.3360):
MAE: 0.7758, RMSE: 1.1483, R2: 0.9769
sklearn Ridge (scaled, alpha=0.3360):
MAE: 0.7758, RMSE: 1.1483, R2: 0.9769

In [31]:

```
best_alpha = 0.336
my_ridge = MyLinearRegression(alpha=best_alpha, fit_intercept=True)
my_ridge.fit(X_train_scaled, Y_train)
y_pred_my = my_ridge.predict(X_test_scaled)
mae_my = mean_absolute_error(Y_test, y_pred_my)
rmse_my = np.sqrt(mean_squared_error(Y_test, y_pred_my))
r2_my = r2_score(Y_test, y_pred_my)
print("MyLinearRegression (scaled, alpha={:.4f}):".format(best_alpha))
print(f"MAE: {mae_my:.4f}, RMSE: {rmse_my:.4f}, R2: {r2_my:.4f}")
from sklearn.linear_model import Ridge
ridge_sk = Ridge(alpha=best_alpha)
ridge_sk.fit(X_train_scaled, Y_train)
y_pred_sk = ridge_sk.predict(X_test_scaled)
mae_sk = mean_absolute_error(Y_test, y_pred_sk)
rmse_sk = np.sqrt(mean_squared_error(Y_test, y_pred_sk))
r2_sk = r2_score(Y_test, y_pred_sk)
print("\nsklearn Ridge (scaled, alpha={:.4f}):".format(best_alpha))
print(f"MAE: {mae_sk:.4f}, RMSE: {rmse_sk:.4f}, R2: {r2_sk:.4f}")
```

MyLinearRegression (scaled, alpha=0.3360):

MAE: 0.7758, RMSE: 1.1483, R2: 0.9769

sklearn Ridge (scaled, alpha=0.3360):

MAE: 0.7758, RMSE: 1.1483, R2: 0.9769

In []: Протестируем собственную модель с масштабированием и подобранным коэффициентом регуляризации.
MyLinearRegression (RobustScaler, alpha=0.336):
MAE: 0.7756, RMSE: 1.1487, R2: 0.9769
Метрики очень близки к полученным ранее, значит, выбросы не оказывают существенного влияния на модель.

In [32]:

```
robust_scaler = RobustScaler()
X_train_robust = robust_scaler.fit_transform(X_train)
X_test_robust = robust_scaler.transform(X_test)
my_ridge_robust = MyLinearRegression(alpha=0.336, fit_intercept=True)
my_ridge_robust.fit(X_train_robust, Y_train)
y_pred_robust = my_ridge_robust.predict(X_test_robust)
mae_robust = mean_absolute_error(Y_test, y_pred_robust)
```



```
rmse_robust = np.sqrt(mean_squared_error(Y_test, y_pred_robust))
r2_robust = r2_score(Y_test, y_pred_robust)
print("MyLinearRegression (RobustScaler, alpha=0.336):")
print(f"MAE: {mae_robust:.4f}, RMSE: {rmse_robust:.4f}, R2: {r2_robust:.4f}")
```

```
MyLinearRegression (RobustScaler, alpha=0.336):
MAE: 0.7756, RMSE: 1.1487, R2: 0.9769
```

In []: *## Выводы:*

В ходе работы были реализованы и рассмотрены различные варианты линейной регрессии. Результаты, это подтвердило, что зависимость целевой переменной от признаков близка к линейной. Полиномиальные признаки было решено не использовать из-за риска переобучения. Моя собственная реализация совпала по результатам с библиотечным аналогом.